

UNIDAD 1.

DESARROLLO DE

SOFTWARE

Entornos de Desarrollo

**1º CFGS Desarrollo de Aplicaciones
Multiplataforma**

Índice

1.	INTRODUCCIÓN	3
2.	SOFTWARE Y PROGRAMAS. TIPOS DE SOFTWARE.....	3
3.	RELACIÓN HARDWARE – SOFTWARE	4
4.	LENGUAJES DE PROGRAMACIÓN.....	6
4.1.	CONCEPTO Y CARACTERÍSTICAS	9
4.2.	LENGUAJES DE PROGRAMACIÓN ESTRUCTURADOS	12
4.3.	LENGUAJES DE PROGRAMACIÓN ORIENTADOS A OBJETOS.....	12
4.4.	MÁQUINAS VIRTUALES.....	14
4.4.1.	Frameworks	14
4.4.2.	Entornos de ejecución	15
4.4.3.	La máquina virtual de Java	16
5.	DESARROLLO DE SOFTWARE	18
5.1.	CICLOS DE VIDA DEL SOFTWARE.....	18
6.	FASES EN EL DESARROLLO Y EJECUCIÓN DEL SOFTWARE.....	21
6.1.	ANÁLISIS.....	22
6.2.	DISEÑO	26
6.3.	CODIFICACIÓN. TIPOS DE CÓDIGO.....	27
6.3.1.	FASES EN LA OBTENCIÓN DEL CÓDIGO	27
6.3.1.2.	Código Objeto	28
6.3.1.3.	Código Ejecutable.....	29
6.4.	PRUEBAS	30
6.5.	DOCUMENTACIÓN	31
6.6.	EXPLOTACIÓN	32
6.7.	MANTENIMIENTO	33
7.	METODOLOGÍAS DE DESARROLLO DE SOFTWARE.....	34
7.1.	Evolución de las metodologías	34
7.2.	Modelo de desarrollo ágil.....	35
	SCRUM.....	36
8.	HERRAMIENTAS UTILIZADAS EN PROGRAMACIÓN.....	38

1. INTRODUCCIÓN

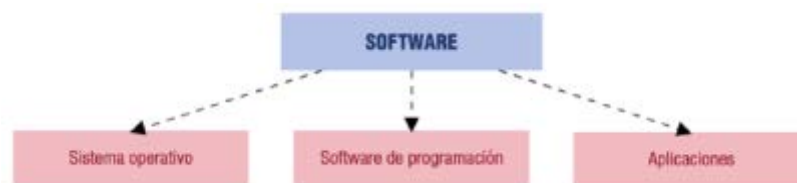
Cuando hablamos de ordenadores, todos hemos escuchado que se componen básicamente de dos partes: el hardware y el software. El **hardware** lo forman los componentes físicos que se pueden ver y tocar: el monitor, el teclado, el ratón, la placa base, la memoria RAM, el microprocesador, las tarjetas de expansión, el disco duro, etc. En cambio, el **software** forma la parte lógica del ordenador que no se puede tocar. Comprende el conjunto de programas y aplicaciones que actúan sobre el hardware del ordenador y facilitan al usuario la realización de diferentes tareas.

En esta unidad nos centraremos en conocer los conceptos básicos que tienen que ver con el desarrollo de software.

2. SOFTWARE Y PROGRAMAS. TIPOS DE SOFTWARE

Como hemos dicho anteriormente, el software es el conjunto de programas informáticos que actúan sobre el hardware para ejecutar lo que el usuario desee.

Según su función se distinguen tres tipos de software: sistema operativo, software de programación y aplicaciones.

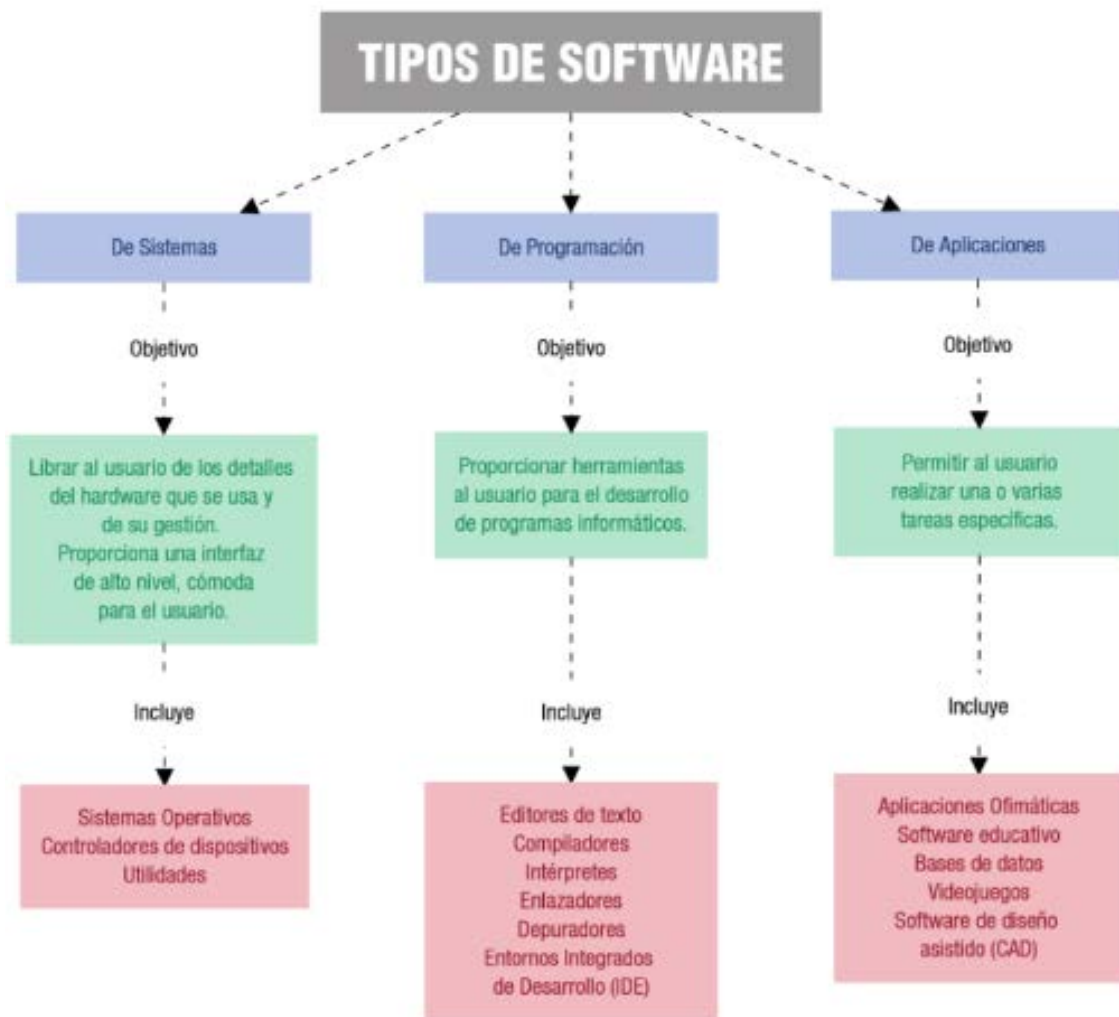


El sistema operativo es el software base que ha de estar instalado y configurado en nuestro ordenador para que las aplicaciones puedan ejecutarse y funcionar. Son ejemplos de sistemas operativos: Windows, Linux, Mac OS X ...

El software de programación es el conjunto de herramientas que nos permiten desarrollar programas informáticos, y las aplicaciones informáticas son un conjunto de programas que tienen una finalidad más o menos concreta. Son ejemplos de aplicaciones: un procesador de textos, una hoja de cálculo, el software para reproducir música, un videojuego, etc.

A su vez, un programa es un conjunto de instrucciones escritas en un lenguaje de programación.

En definitiva, distinguimos los siguientes tipos de software:



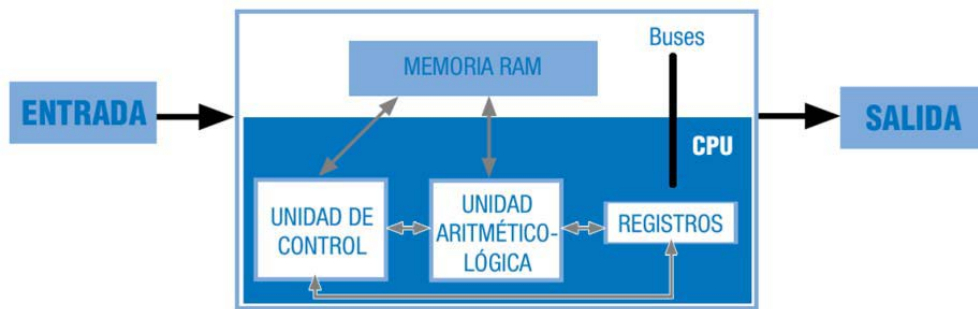
3. RELACIÓN HARDWARE – SOFTWARE

Como ya hemos comentado, al conjunto de dispositivos físicos que conforman un ordenador se le denomina hardware.

Existe una relación indisoluble entre éste y el software, ya que ambos necesitan estar instalados y configurados correctamente para que el equipo funcione.

El software se ejecutará sobre los dispositivos físicos.

La primera arquitectura hardware con programa almacenado se estableció en 1946 por John Von Neumann:



Esta relación software-hardware la podemos poner de manifiesto desde dos puntos de vista:

- **Desde el punto de vista del sistema operativo**

El sistema operativo es el encargado de coordinar al hardware durante el funcionamiento del ordenador, actuando como intermediario entre éste y las aplicaciones que están corriendo en un momento dado.

Todas las aplicaciones necesitan recursos hardware durante su ejecución (tiempo de CPU, espacio en memoria RAM, tratamiento de interrupciones, gestión de los dispositivos de Entrada/Salida, etc.). Será siempre el sistema operativo el encargado de controlar todos estos aspectos de manera "oculta" para las aplicaciones (y para el usuario).

- **Desde el punto de vista de las aplicaciones**

Ya hemos dicho que una aplicación no es otra cosa que un conjunto de programas, y que éstos están escritos en algún lenguaje de programación que el hardware del equipo debe interpretar y ejecutar.

Hay multitud de lenguajes de programación diferentes (como ya veremos en su momento). Sin embargo, todos tienen algo en común: estar escritos con sentencias de un idioma que el ser humano puede aprender y usar fácilmente. Por otra parte, el hardware de un ordenador sólo es capaz de interpretar señales eléctricas (ausencias o presencias de tensión) que, en informática, se traducen en secuencias de 0 y 1 (código binario).

Esto nos hace plantearnos una cuestión: ¿Cómo será capaz el ordenador de "entender" algo escrito en un lenguaje que no es el suyo?

Como veremos a lo largo de esta unidad, tendrá que pasar algo (un proceso de traducción de código) para que el ordenador ejecute las instrucciones escritas en un lenguaje de programación.

4. LENGUAJES DE PROGRAMACIÓN

Ya comentamos anteriormente que los programas informáticos están escritos usando algún lenguaje de programación, por lo que podemos definir un **Lenguaje de Programación** como un idioma creado de forma artificial, formado por un conjunto de símbolos y normas que se aplican sobre un alfabeto para obtener un código, que el hardware del ordenador puede entender y ejecutar.

Los lenguajes de programación son los que nos permiten comunicarnos con el hardware del ordenador.

Hay multitud de lenguajes de programación, cada uno con unos símbolos y unas estructuras diferentes. Además, cada lenguaje está enfocado a la programación de tareas o áreas determinadas. Por ello, la elección del lenguaje a utilizar en un proyecto es una cuestión de extrema importancia.

Los lenguajes de programación han sufrido su propia evolución, como se puede apreciar en la figura siguiente:



Características de los Lenguajes de Programación

- Lenguaje máquina:
 - Sus instrucciones son combinaciones de unos y ceros.
 - Es el único lenguaje que entiende directamente el ordenador. (No necesita traducción).
 - Fue el primer lenguaje utilizado.
 - Es único para cada procesador (no es portable de un equipo a otro).
 - Nadie puede programar en este lenguaje.

```
*R PC=FFFF SP=FF A=FF B=FF PSM=FF DPTR=FFFF
R0=FF R1=FF R2=FF R3=FF R4=FF R5=FF R6=FF R7=FF
*D 2050,2100
2050 01 02 04 08 10 20 40 80 55 AA 33 01 02 04 08 10 ..... @.U.3.....
2060 20 40 80 55 AA 33 01 02 04 08 10 20 40 80 55 AA @.U.3..... @.U.
2070 33 01 02 04 08 10 20 40 80 55 AA 33 01 02 04 08 3..... @.U.3.....
2080 10 20 40 80 55 AA 33 01 02 04 08 10 20 40 80 55 . @.U.3..... @.U
2090 AA 33 01 02 04 08 10 20 40 80 55 AA 33 01 02 04 .3..... @.U.3...
20A0 08 10 20 40 80 55 AA 33 01 02 04 08 10 20 40 80 .. @.U.3..... @.
20B0 55 AA 33 01 02 04 08 10 20 40 80 55 AA 33 01 02 U.3..... @.U.3..
20C0 04 08 10 20 40 80 55 AA 33 01 02 04 08 10 20 40 ... @.U.3..... @
20D0 80 55 AA 33 01 02 04 08 10 20 40 80 55 AA 33 01 .U.3..... @.U.3.
20E0 02 04 08 10 20 40 80 55 AA 33 01 02 04 08 10 20 .... @.U.3.....
20F0 40 80 55 AA 33 01 02 04 08 10 20 40 80 55 AA 33 @.U.3..... @.U.3
2100 01 02 04 08 10 20 40 80 55 AA 33 01 02 04 08 10 ..... @.U.3.....
*
```

- Lenguaje ensamblador:
 - o Sustituyó al lenguaje máquina para facilitar la labor de programación.
 - o En lugar de unos y ceros se programa usando mnemotécnicos (instrucciones complejas).
 - o Necesita traducción al lenguaje máquina para poder ejecutarse.
 - o Sus instrucciones son sentencias que hacen referencia a la ubicación física de los archivos en el equipo.
 - o Es difícil de utilizar.

```
[0x00000000]> pd
0x00000000  90          nop
0x00000001  90          nop
0x00000002  6800009c00  push 0x9c0000 ; 0x009c0000
0x00000007  e8c7ace37b  call 0x7be3acd3
0x7be3acd3(unk)
0x0000000c  bb04009c00  mov ebx, 0x9c0004
0x00000011  8903        mov [ebx], eax
0x00000013  e81903f47b  call 0x7bf40331
0x7bf40331( )
0x00000018  bb08009c00  mov ebx, 0x9c0008
0x0000001d  8903        mov [ebx], eax
0x0000001f  bb00009c00  mov ebx, 0x9c0000
0x00000024  c60300      mov byte [ebx], 0x0
-> 0x00000027  68e8030000  push 0x3e8 ; 0x000003e8
0x0000002c  e81124e37b  call 0x7be32442
0x7be32442(unk)
<= 0x00000031  ebf4        jmp 0x100000027
0x00000033  90          nop
0x00000034  ff          invalid
0x00000035  ff          invalid
0x00000036  ff          invalid
0x00000037  ff          invalid
```

- Lenguaje de alto nivel basados en código:
 - o Sustituyeron al lenguaje ensamblador para facilitar más la labor de programación.
 - o En lugar de mnemotécnicos, se utilizan sentencias y órdenes derivadas del idioma inglés. (Necesita traducción al lenguaje máquina).
 - o Son más cercanos al razonamiento humano.
 - o Son utilizados hoy día, aunque la tendencia es que cada vez menos.

```
83 preg_match_all("/<a[^\>]*\".href=[\"|'](.*)[\"|']
84 $enlaces = $enlaces[1];
85
86 foreach($enlaces as $enlace){
87     $enlace = substr($enlace, 0, -4);
88
89     if (substr($enlace, 0, 5)=="http:"){
90         echo "";
91     } else
92     {
93         $detecta = consulta_simple("SELECT * FROM
94         if ($detecta['palabra']=='')
95         {echo "Una URL es incorrecta en Relacio
96         echo $enlace."<br><br><br>";
97     };
98 }
```

- Lenguajes visuales:
 - Están sustituyendo a los lenguajes de alto nivel basados en código.
 - En lugar de sentencias escritas, se programa gráficamente usando el ratón y diseñando directamente la apariencia del software.
 - Su correspondiente código se genera automáticamente.
 - Necesitan traducción al lenguaje máquina.
 - Son completamente portables de un equipo a otro.



4.1. CONCEPTO Y CARACTERÍSTICAS

Ya sabemos que los lenguajes de programación han evolucionado, y siguen haciéndolo, siempre hacia la mayor usabilidad de estos (que el mayor número posible de usuarios lo utilicen y exploten).

La elección del lenguaje de programación para codificar un programa dependerá de las características del problema a resolver.

Concepto

Un lenguaje de programación es un conjunto de caracteres, las reglas para combinarlos y las reglas que definen qué ocurre cuando los ejecuta un ordenador.

Se compone de:

- Alfabeto: conjunto de símbolos permitidos.
- Sintaxis: normas de construcción permitidas de los símbolos del lenguaje.
- Semántica: significado de las construcciones para hacer acciones válidas.

Características

Podemos clasificar los distintos tipos de Lenguajes de Programación en base a distintas características:

1. Según el nivel de abstracción:

- Lenguajes de Programación **De alto nivel**: por su esencia, están más próximos al razonamiento humano.
 - o Son más fáciles de aprender porque utilizan palabras del lenguaje natural.
 - o Son independientes de la máquina donde se ejecuten.
 - o Deben traducirse o compilarse en instrucciones que el ordenador pueda entender.
- Lenguajes de Programación **De medio nivel**: están más próximos al funcionamiento interno de la computadora:
 - o Tienen características de lenguajes de bajo y alto nivel.
 - o Suelen utilizarse, entre otras cosas, para crear sistemas operativos.
 - o C es el lenguaje más utilizado de este tipo.
- Lenguajes de Programación **De bajo nivel**: están más próximos al funcionamiento interno de la computadora:
 - o El de más bajo nivel, es el lenguaje máquina, formados por 0 y 1, y son específicos de cada procesador.
 - o El lenguaje ensamblador es una primera abstracción del lenguaje máquina y usa nemónicos para identificar instrucciones y datos. Se traduce a lenguaje máquina para poder ejecutarse.

2.Según el paradigma de programación:

Un paradigma de programación define un conjunto de reglas, patrones y estilos de programación que usan los lenguajes de programación. Es decir, la filosofía utilizada en el desarrollo. Un lenguaje puede usar más de un paradigma.

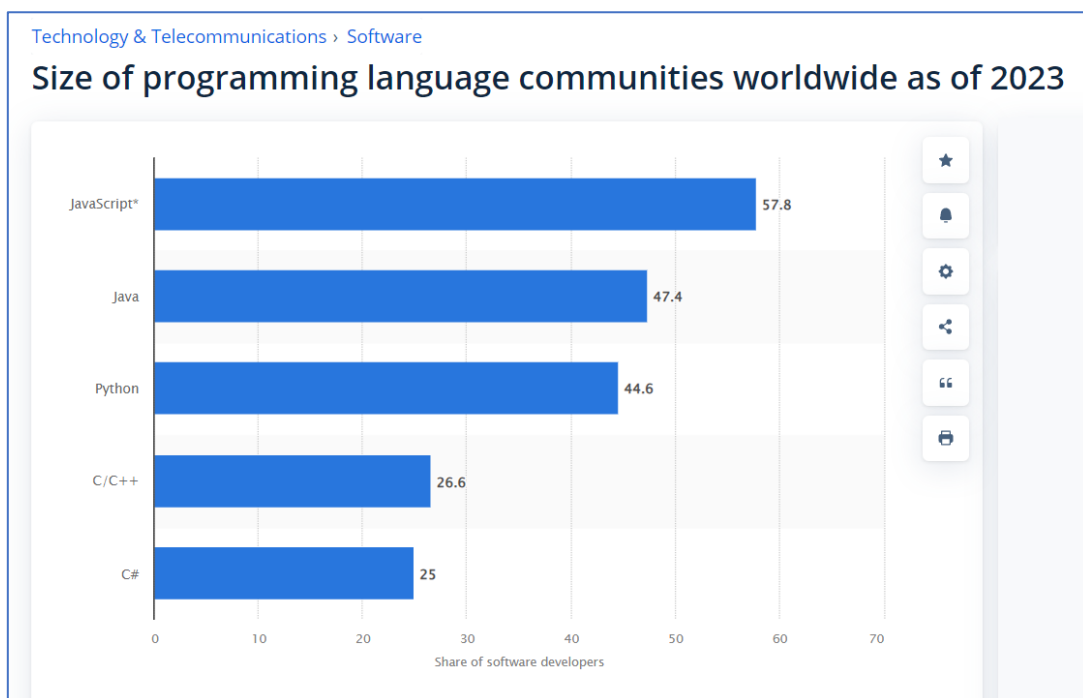
- Paradigma **imperativo**: Parte de los lenguajes máquinas donde las instrucciones eran muy simples. Estos lenguajes consisten en unas sentencias que establecen cómo se deben manipular la información contenida en memoria y cómo se debe recoger o enviar información hacia los dispositivos. La sentencia principal es la asignación, y con las instrucciones de control se establece el orden y se cambia el flujo. Dentro de esta categoría están:
 - Paradigma **estructurado**:
 - Se estructuran en 3 construcciones lógicas: estructura secuencial, condicional y repetitiva.
 - Todo el código se concentra en un único bloque, que puede ser leído secuencialmente de principio a fin.
 - Paradigma **modular**:
 - Es la división de un programa estructurado en partes más manejables denominadas módulos.
 - Cada módulo tiene una entrada y una salida.
 - La comunicación entre módulos está perfectamente controlada y se trabaja de forma independiente en cada módulo, que además, puede ser reutilizado.
 - Paradigma **orientado a objetos**:
 - Un programa está compuesto por un conjunto de objetos, y no por un conjunto de instrucciones.
 - Un objeto consta de una estructura de datos (atributos), y de las operaciones que definen el comportamiento del objeto y cambian el valor de los atributos. Los objetos se comunican mediante el paso de mensajes.
 - Se facilita la reutilización de código, el trabajo en equipo, y el mantenimiento, pero es más complejo.
- Paradigma **funcional**:
 - Basado en el concepto matemático de función.
 - El valor generado por una función, depende de los argumentos de entrada.
 - No hay existencia de asignaciones de variables.
 - Se indica lo que se tiene que obtener, no cómo obtenerlo.
- Paradigma **de tiempo real**:
 - Se utiliza para programación en tiempo real, donde se establecen plazos que deben ser cumplidos.
- Paradigma **concurrente**:
 - Permiten realizar paralelismo (ejecución al mismo tiempo) entre tareas o procesos.
 - Es importante la sincronización entre procesos, y el acceso a recursos compartidos.

3. Según la forma de ejecución.

- Lenguajes **compilados**:
 - Los compiladores son programas que traducen un programa (fuente) escrito en un lenguaje de alto nivel a un código que pueda ejecutar la máquina directamente.
- Lenguajes **interpretados**:
 - Los intérpretes son otra alternativa para traducir los programas escritos en un lenguaje de alto nivel.
 - Cada vez que se ejecuta una instrucción, se debe interpretar y traducir al lenguaje máquina.
 - Los lenguajes interpretados, se ejecutan más lentos que los compilados, pero no hay que compilar el programa fuente después de cada modificación.

Java se traduce primero en un formato intermedio, llamado *bytecode*, y después una máquina virtual los interpreta.

A pesar de la inmensa cantidad de lenguajes de programación existentes, Java, JavaScript, C, C++, y Python concentran alrededor del 50% del interés de la comunidad informática mundial.



Fuente: <https://www.statista.com/statistics/1241923/worldwide-software-developer-programming-language-communities/>

4.2. LENGUAJES DE PROGRAMACIÓN ESTRUCTURADOS

Aunque los requerimientos actuales de software son bastante más complejos de lo que la técnica de programación estructurada es capaz, es necesario por lo menos conocer las bases de los Lenguajes de Programación estructurados, ya que a partir de ellos se evolucionó hasta otros lenguajes y técnicas más completas (orientada a eventos u objetos) que son las que se usan actualmente.

La programación estructurada se define como una técnica para escribir lenguajes de programación que permite sólo el uso de tres tipos de sentencias o estructuras de control:

- Sentencias secuenciales.
- Sentencias selectivas (condicionales).
- Sentencias repetitivas (iteraciones o bucles).

Los lenguajes de programación que se basan en la programación estructurada reciben el nombre de lenguajes de programación estructurados.

La programación estructurada fue de gran éxito por su sencillez a la hora de construir y leer programas. Fue sustituida por la programación modular, que permitía dividir los programas grandes en trozos más pequeños (siguiendo la conocida técnica "divide y vencerás"). A su vez, luego triunfaron los lenguajes orientados a objetos y de ahí a la programación visual (siempre es más sencillo programar gráficamente que en código).

VENTAJAS

- Los programas son fáciles de leer, sencillos y rápidos.
- El mantenimiento de los programas es sencillo.
- La estructura del programa es sencilla y clara.

INCONVENIENTES

- Todo el programa se concentra en un único bloque (si se hace demasiado grande es difícil manejarlo).
- No permite reutilización eficaz de código, ya que todo va "en uno". Es por esto por lo que la programación estructurada sustituyó a la programación modular, donde los programas se codifican por módulos y bloques, permitiendo mayor funcionalidad.

Ejemplos de lenguajes estructurados: Pascal, C, Fortran.

La Programación estructurada evolucionó hacia la Programación modular, que divide el programa en trozos de código llamados módulos con una funcionalidad concreta, que podrán ser reutilizables.

4.3. LENGUAJES DE PROGRAMACIÓN ORIENTADOS A OBJETOS

Después de comprender que la programación estructurada no es útil cuando los programas se hacen muy largos, es necesaria otra técnica de programación que solucione este inconveniente. Nace así la Programación Orientada a Objetos (en adelante, P.O.O.).

Los lenguajes de programación orientados a objetos tratan a los programas no como un conjunto ordenado de instrucciones (tal como sucedía en la programación estructurada) sino como un conjunto de objetos que colaboran entre ellos para realizar acciones.

En la P.O.O. los programas se componen de objetos independientes entre sí que colaboran para realizar acciones.

Los objetos son reutilizables para proyectos futuros.

Su primera desventaja es clara: no es una programación tan intuitiva como la estructurada.

A pesar de eso, alrededor del 55% del software que producen las empresas se hace usando esta técnica.

Razones:

- El código es reutilizable.
- Si hay algún error, es más fácil de localizar y depurar en un objeto que en un programa entero.

Características:

- Los objetos del programa tendrán una serie de atributos que los diferencian unos de otros.
- Se define clase como una colección de objetos con características similares.
- Mediante los llamados métodos, los objetos se comunican con otros produciéndose un cambio de estado de estos.
- Los objetos son, pues, como unidades individuales e indivisibles que forman la base de este tipo de programación.

Principales lenguajes orientados a objetos: Ada, C++, VB.NET, Delphi, Java, PowerBuilder, etc.

4.4. MÁQUINAS VIRTUALES

Una máquina virtual es una aplicación software de una máquina (por ejemplo, un ordenador) que ejecuta los programas como si fuese una máquina real. Se pueden clasificar en dos categorías:

- Máquinas virtuales de sistema (System Virtual Machine). Permiten ejecutar en la misma máquina física varias máquinas virtuales, cada una con un sistema operativo, permitiendo coexistir diferentes sistemas operativos sobre una misma máquina. Ejemplos de software son: VMWare Workstation y Virtual Box.
- Máquinas virtuales de proceso (Process Virtual Machine). Se les llama en ocasiones, “máquina virtual de aplicación”, se ejecuta como un proceso normal dentro de un sistema operativo y soporta un solo proceso. La máquina se inicia automáticamente cuando se lanza el proceso que se desea ejecutar y se detiene cuando finaliza. Su objetivo es proporcionar un entorno de ejecución independiente de la plataforma hardware y del sistema operativo, que oculte los detalles de la plataforma subyacente y permita que un programa se ejecute siempre de la misma forma sobre cualquier plataforma. El ejemplo más conocido actualmente de este tipo de máquina virtual es la máquina virtual de Java.

4.4.1. Frameworks

Un framework es una estructura de ayuda al programador, en base a la cual podemos desarrollar proyectos sin partir desde cero.

Se trata de una plataforma software donde están definidos programas soporte, bibliotecas, lenguaje interpretado, etc., que ayuda a desarrollar y unir los diferentes módulos o partes de un proyecto.

Con el uso de framework podemos pasar más tiempo analizando los requerimientos del sistema y las especificaciones técnicas de nuestra aplicación, ya que la tarea laboriosa de los detalles de programación queda resuelta.

- Ventajas de utilizar un framework:
 - Desarrollo rápido de software.
 - Reutilización de partes de código para otras aplicaciones.
 - Diseño uniforme del software.
 - Portabilidad de aplicaciones de un ordenador a otro, ya que los bytecodes que se generan a partir del lenguaje fuente podrán ser ejecutados sobre cualquier máquina virtual.
- Inconvenientes:
 - Gran dependencia del código respecto al framework utilizado (sin cambios de framework, habrá que reescribir gran parte de la aplicación).
 - La instalación e implementación del framework en nuestro equipo consume bastantes recursos del sistema.

Ejemplos de Frameworks:

- .NET es un framework para desarrollar aplicaciones sobre Windows. Ofrece “Visual Studio .NET” que nos da facilidades para construir aplicaciones, y su motor es el “.NET Framework” que permite ejecutar dichas aplicaciones. Es un componente que se instala sobre el sistema operativo.
- Spring de Java. Son conjuntos de bibliotecas (API's) para el desarrollo y ejecución de aplicaciones.

4.4.2. Entornos de ejecución

Un entorno de ejecución es un servicio de máquina virtual que sirve como base software para la ejecución de programas. En ocasiones pertenece al propio sistema operativo, pero también se puede instalar como software independiente que funcionará por debajo de la aplicación. Es decir, es un conjunto de utilidades que permiten la ejecución de programas.

Durante la ejecución, los entornos se encargarán de:

- Configurar la memoria principal disponible en el sistema.
- Enlazar los archivos del programa con las bibliotecas existentes y con los subprogramas creados. Considerando que:
 - o **Bibliotecas** son el conjunto de subprogramas que sirven para desarrollar o comunicar componentes software pero que ya existen previamente. Ejemplos:
 - Biblioteca estándar del lenguaje C
 - Entrada y salida por terminal
 - Manejo de archivos
 - Funciones matemáticas
 - Biblioteca estándar del lenguaje Java
 - Entrada y salida por terminal
 - Manejo de archivos
 - Funciones matemáticas
 - Interfaz gráfica
 - Red
 - Bases de datos
 - Gráficos (sólo 2D)
 - o Cada biblioteca está compuesta por varios archivos objeto. Tipos:
 - Bibliotecas dinámicas (.DLL o .so) (.jar en Java)
 - Bibliotecas estáticas (.LIB ó .a)
 - o **Subprogramas** serán aquellos que hemos creado a propósito para el programa.

Funcionamiento del entorno de ejecución

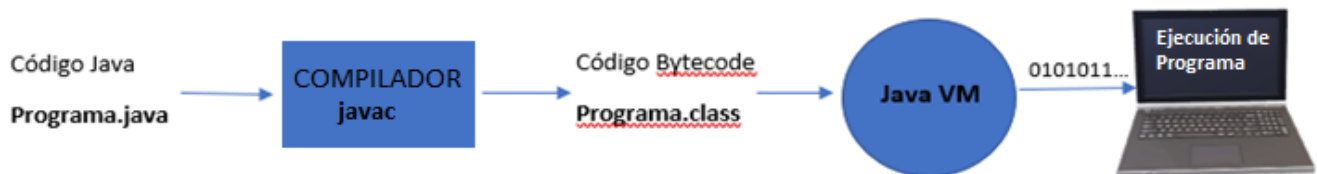
El entorno de ejecución está formado por la máquina virtual y los APIs (bibliotecas de clases estándar, necesarias para que la aplicación, escrita en algún lenguaje de programación, pueda ser ejecutada). Estos dos componentes se suelen distribuir conjuntamente, porque necesitan ser compatibles entre sí.

El entorno funciona como intermediario entre el lenguaje fuente y el sistema operativo, y consigue ejecutar aplicaciones.

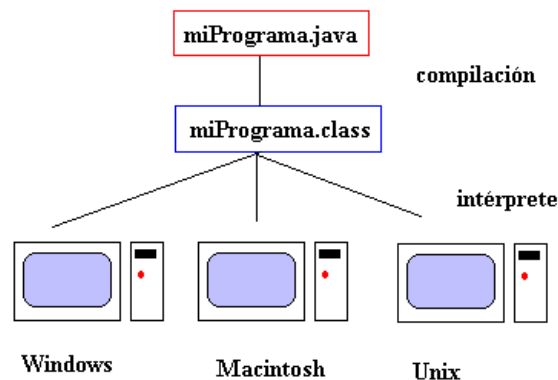
Sin embargo, si lo que queremos es desarrollar nuevas aplicaciones, no es suficiente con el entorno de ejecución, necesitaríamos de un entorno de desarrollo (se verá en la siguiente unidad).

4.4.3. La máquina virtual de Java

Los programas compilados en lenguaje Java se pueden ejecutar en cualquier plataforma: es decir, pueden ejecutarse en entornos UNIX, Mac, Windows, Solaris, etc. Esto es debido a que el código generado por el compilador no lo ejecuta el procesador del ordenador, sino que lo ejecuta la Máquina Virtual de Java (JVM, Java Virtual Machine).



El código fuente del programa está escrito en fichero de texto plano que tienen la extensión *.java*. La compilación del fichero, mediante el compilador Java **javac**, genera un fichero (o varios) con la extensión *.class* (siempre y cuando no haya errores). Un fichero *.class* contiene código en un lenguaje intermedio cercano al lenguaje máquina, pero independiente del ordenador y el sistema operativo que se ejecuta, este código se llama *bytecode*. La Máquina Virtual de Java toma y traduce el bytecode en código binario para el procesador que se utiliza para ejecutar el programa. Como está en muchos sistemas operativos diferentes, los ficheros *.class* pueden ser ejecutados en distintas plataformas: Microsoft Windows, Solaris, Linux o Mac OS.



Las tareas principales de la *Máquina Virtual de Java* son:

- Reservar espacio en memoria para los objetos creados y liberar la memoria no utilizada.
- Comunicarse con el sistema huésped (sistema donde se ejecuta la aplicación) para ciertas funciones, como controlar el acceso a dispositivos hardware.
- Vigilar en cumplimiento de las normas de seguridad de las aplicaciones Java.

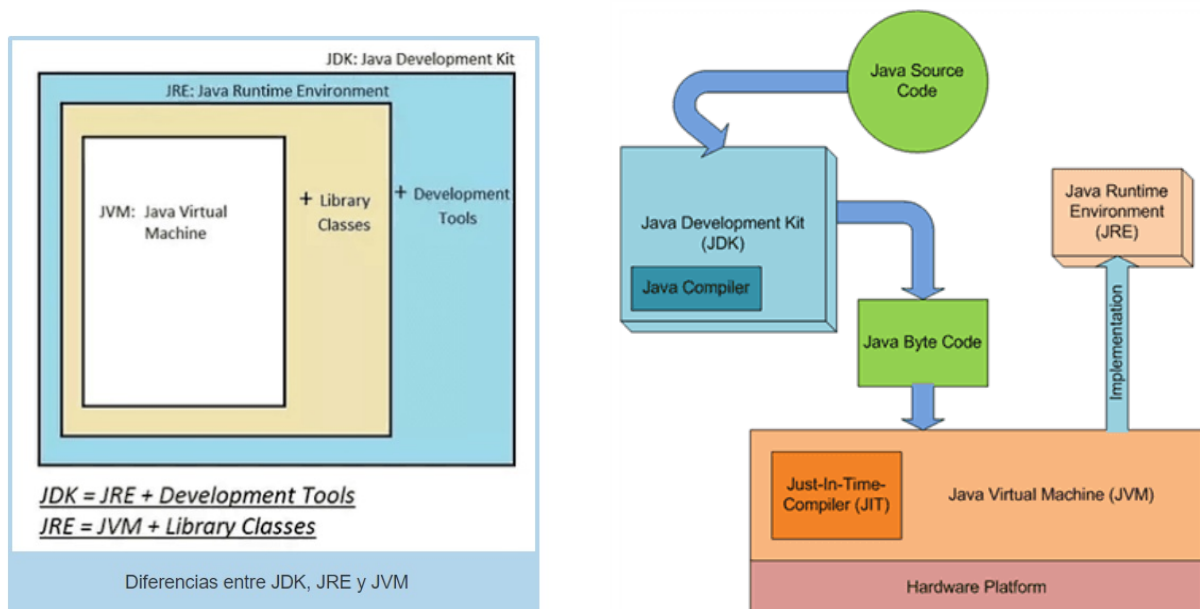
La principal desventaja de los lenguajes basados en máquina virtual es que son más lentos que los lenguajes completamente compilados, debido a la sobrecarga que genera tener una capa de software intermedia entre la aplicación y el hardware del ordenador.

Para instalar el entorno de ejecución de la *Máquina Virtual Java* hemos de acceder a la web de Oracle (<https://www.oracle.com/java/technologies/downloads>) y descargar el fichero que nos interese según el sistema operativo del ordenador donde vayamos a instalarlo.

El **JRE** se compone de un conjunto de utilidades que permitirá la ejecución de programas java sobre cualquier tipo de plataforma.

JRE está formado por:

- Una Máquina Virtual Java, para la interpretación del código de la aplicación escrito en Java.
- Bibliotecas de clase estándar que implementan el API de Java.



JDK – Java Development Kit: es la herramienta que usan los desarrolladores. Proporciona el entorno para desarrollar y ejecutar el programa Java. Incluye dos cosas:

1. Herramientas de desarrollo (para proporcionar un entorno para desarrollar sus programas de Java)
2. JRE (para ejecutar su programa java).

JRE – Java Runtime Environment (JRE): proporciona un entorno para ejecutar (no desarrollar) el programa Java en su máquina. JRE solo lo utilizan aquellos que solo desean ejecutar los Programas Java, es decir, los usuarios finales de su sistema.

5. DESARROLLO DE SOFTWARE

Entendemos por Desarrollo de Software todo el proceso que ocurre desde que se concibe una idea hasta que un programa está implementado en el ordenador y funcionando.



El proceso de desarrollo, que un principio puede parecer una tarea simple, consta de una serie de pasos de obligado cumplimiento, pues solo así podremos garantizar que los programas creados son eficientes, fiables, seguros y responden a las necesidades de los usuarios finales (aquellos que van a utilizar el programa).

El desarrollo de software es un proceso que conlleva una serie de pasos, que llamaremos ciclos del software.

5.1. CICLOS DE VIDA DEL SOFTWARE

La construcción de software es un proceso que puede llegar a ser muy complejo y que exige gran coordinación y disciplina del grupo de trabajo que lo desarrolle.

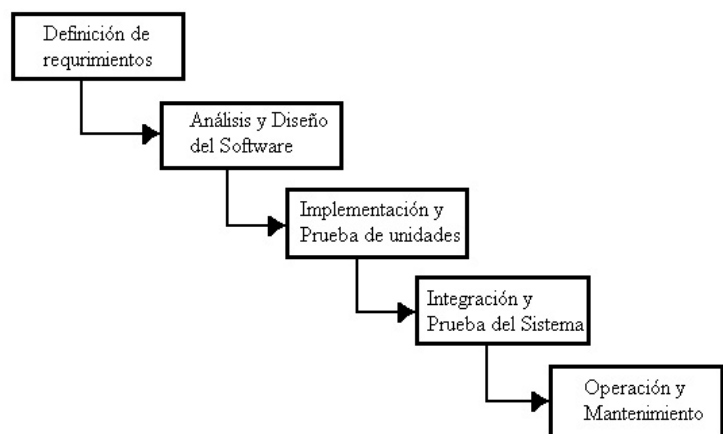
La serie de pasos a seguir para desarrollar un programa es lo que se conoce como Ciclo de Vida del Software. Siempre se debe aplicar un modelo de ciclo de vida al desarrollo de cualquier proyecto software.

En función del autor que se lea, se pueden encontrar diferentes modelos de ciclos de vida, pero los más conocidos y utilizados son los que aparecen a continuación:

- **Modelo en Cascada**

También llamado modelo clásico o modelo tradicional.

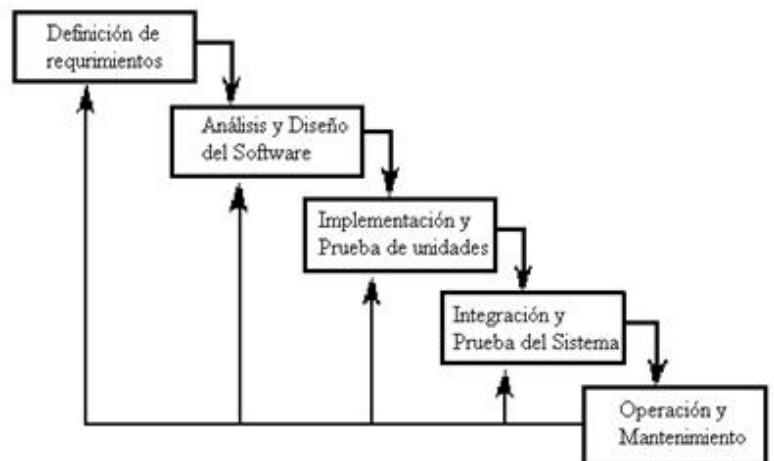
Es prácticamente imposible que se pueda utilizar, ya que requiere conocer de antemano todos los requisitos del sistema. Sólo es aplicable a pequeños desarrollos, ya que las etapas pasan de una a otra sin retorno posible. (se presupone que no habrá errores ni variaciones del software).



- **Modelo en Cascada con Realimentación**

Es uno de los modelos más utilizados. Proviene del modelo anterior, pero se introduce una realimentación entre etapas, de forma que podamos volver atrás en cualquier momento para corregir, modificar o depurar algún aspecto. No obstante, si se prevén muchos cambios durante el desarrollo no es el modelo más idóneo.

Es el modelo perfecto si el proyecto es rígido (pocos cambios, poco evolutivo) y los requisitos están claros.



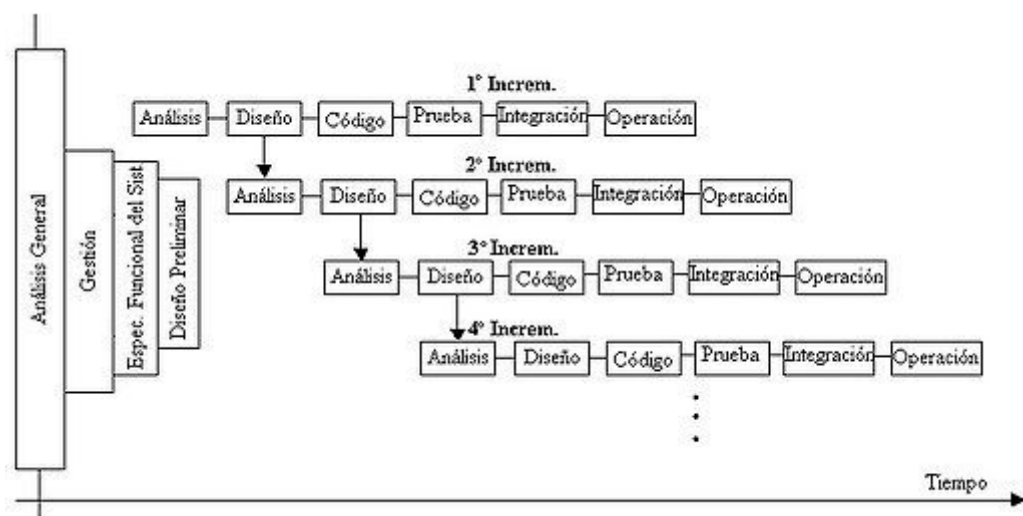
- **Modelos Evolutivos**

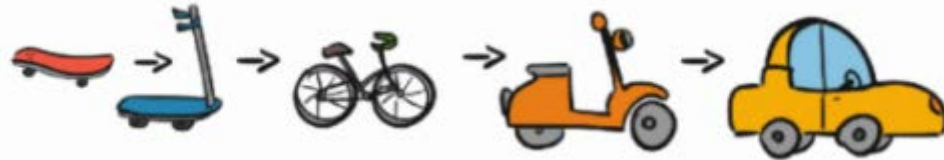
Son más modernos que los anteriores. Tienen en cuenta la naturaleza cambiante y evolutiva del software, entienden que es normal que los requisitos del usuario y del software cambien según se desarrolla el mismo.

Distinguimos dos variantes:

- **Modelo Iterativo Incremental**

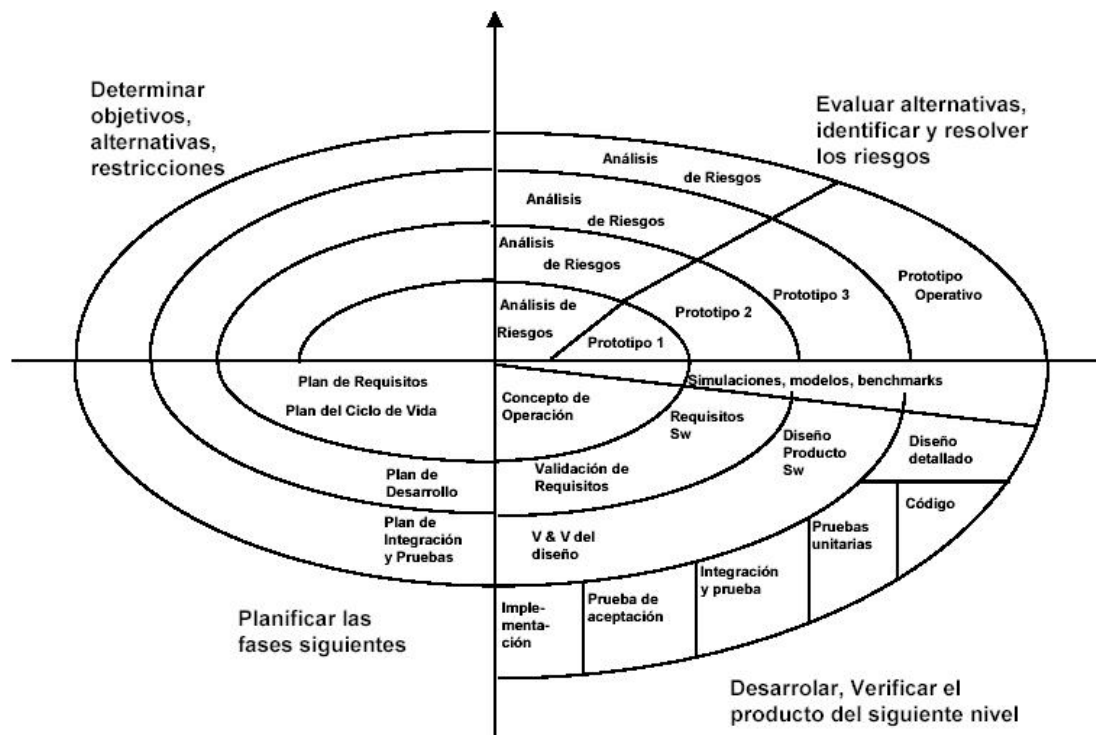
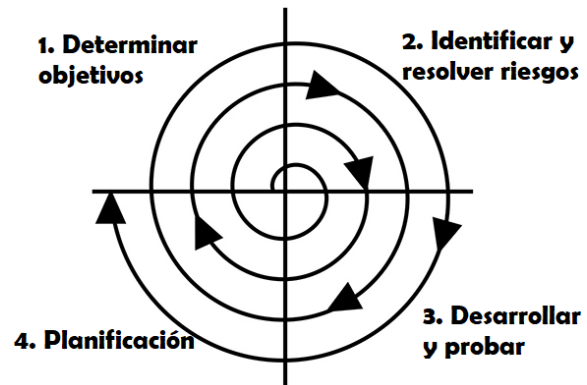
Está basado en el modelo en cascada con realimentación, donde las fases se repiten y refinan, y van propagando su mejora a las fases siguientes. El modelo integral entrega el software en partes pequeñas, pero utilizables, llamadas “incrementos”. En general, cada incremento se construye sobre aquel que ya ha sido entregado.





0 Modelo en Espiral

Es una combinación del modelo anterior con el modelo en cascada. En él, el software se va construyendo repetidamente en forma de versiones que son cada vez mejores, debido a que incrementan la funcionalidad en cada versión. Es un modelo bastante complejo.



6. FASES EN EL DESARROLLO Y EJECUCIÓN DEL SOFTWARE

Ya hemos visto en puntos anteriores que debemos elegir un modelo de ciclo de vida para el desarrollo de nuestro software.

Independientemente del modelo elegido, siempre hay una serie de etapas que debemos seguir para construir software fiable y de calidad.

Estas etapas son:

1. ANÁLISIS DE REQUISITOS.

Se especifican los requisitos funcionales y no funcionales del sistema.

2. DISEÑO.

Se divide el sistema en partes y se determina la función de cada una.

3. CODIFICACIÓN.

Se elige un Lenguajes de Programación y se codifican los programas.

4. PRUEBAS.

Se prueban los programas para detectar errores y se depuran.

5. DOCUMENTACIÓN.

De todas las etapas, se documenta y guarda toda la información.

6. EXPLOTACIÓN.

Instalamos, configuramos y probamos la aplicación en los equipos del cliente.

7. MANTENIMIENTO.

Se mantiene el contacto con el cliente para actualizar y modificar la aplicación el futuro.

6.1. ANÁLISIS

Esta es la primera fase del proyecto. Una vez finalizada, pasamos a la siguiente (diseño).

Es la fase de mayor importancia en el desarrollo del proyecto y todo lo demás dependerá de lo bien detallada que esté. También es la más complicada, ya que no está automatizada y depende en gran medida del analista que la realice.



Es la primera etapa del proyecto, la más complicada y la que más depende de la capacidad del analista.

¿Qué se hace en esta fase?

Se especifican y analizan los requisitos funcionales y no funcionales del sistema.

Requisitos:

- **Funcionales:** Describen con detalle la función que realiza el sistema, cómo reacciona ante determinadas entradas, cómo se comporta en situaciones particulares, etc.
- **No funcionales:** Tratan sobre las características del sistema, como puede ser la fiabilidad, mantenibilidad, sistema operativo, plataforma hardware, restricciones, limitaciones, etc.

Lo fundamental es la buena comunicación entre el analista y el cliente para que la aplicación que se va a desarrollar cumpla con sus expectativas.

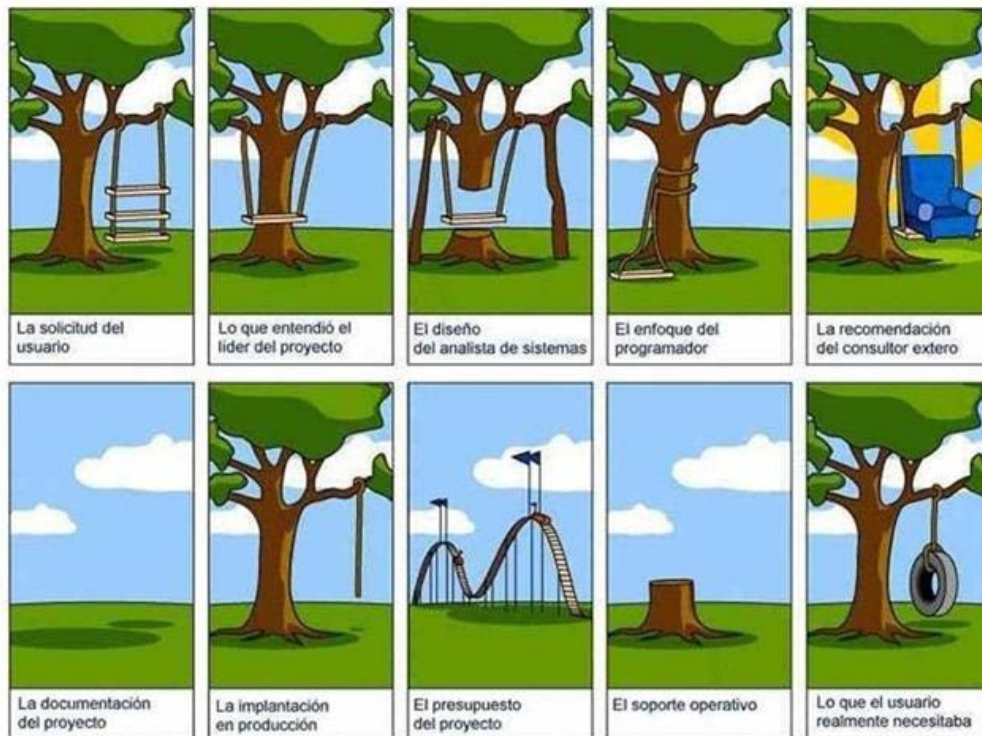
La culminación de esta fase es el documento ERS (Especificación de Requisitos Software).

En este documento quedan especificados:

- La planificación de las reuniones que van a tener lugar.
- Relación de los objetivos del usuario cliente y del sistema.
- Relación de los requisitos funcionales y no funcionales del sistema.
- Relación de objetivos prioritarios y temporización.
- Reconocimiento de requisitos mal planteados o que conllevan contradicciones, etc.

Como ejemplo de requisitos funcionales, en la aplicación para nuestros clientes de las tiendas de cosmética, habría que considerar:

- Si desean que la lectura de los productos se realice mediante códigos de barras.
- Si van a detallar las facturas de compra y de qué manera la desean.
- Si los trabajadores de las tiendas trabajan a comisión, tener información de las ventas de cada uno.
- Si van a operar con tarjetas de crédito.
- Si desean un control del stock en almacén.



TÉCNICAS DE OBTENCIÓN DE REQUISITOS

Para la obtención de estos requisitos se pueden usar las siguientes técnicas de obtención de requisitos:

- Entrevistas: Consiste en hablar con el cliente
- Desarrollo conjunto de aplicaciones: Es un tipo de entrevista muy estructurada aplicable a grupos de personas. Cada persona juega un rol concreto.
- Planificación conjunta de requisitos: Subconjunto de la anterior, dirigidas a la alta dirección que sirve para obtener los requisitos de alto nivel o estratégicos.
- Brainstorming: Reuniones en grupo cuyo objetivo es generar ideas desde diferentes puntos de vista para la resolución de un problema.
- Prototipos: Es una versión inicial del sistema que se utiliza para clarificar algunos puntos o demostrar los conceptos.
- Casos de uso: Es la técnica definida en UML (Unified Modeling Language) que se basa en la representación de escenarios que describen el comportamiento deseado del sistema. Describen qué hace el sistema, no cómo lo hace.



© Scott Adams, Inc./Dist. by UFS, Inc.

sg.com.mx/sgvirtual

2

TÉCNICAS DE REPRESENTACIÓN DE REQUISITOS

Las técnicas de representación de requisitos más habituales son:

- Diagrama de flujo de datos (DFD): Representa el flujo de datos entre los distintos procesos, entidades externas y almacenes que forman el sistema.
- Diagrama de flujo de control (DFC): Similar al DFD, pero con la diferencia de que muestra el flujo de control.
- Diagrama de transición de estados (DTE): Representa cómo se comporta el sistema como consecuencia de sucesos externos.
- Diagrama Entidad /Relación (E-R): Se usa para representar los datos y la forma en la que se relacionan entre ellos.
- Diccionario de datos (DD): Descripción detallada de los datos utilizados por el sistema que gráficamente se encuentran representados en el resto de diagramas.

Una vez terminas esta fase, toda esta información debe quedar recogida, en el documento de **Especificación de Requisitos del Software (ERS)**. Este documento:

- No debe tener ambigüedades
- Debe ser completo
- Debe ser consistente
- Tiene que ser fácil de verificar y modificar
- Tiene que ser fácil de utilizar en la fase de explotación y mantenimiento
- Debe ser fácil de identificar el origen y las consecuencias de los requisitos.
- Sigue como entrada para la siguiente fase en el desarrollo de la aplicación.

Así, según la norma IEEE 830, la ERS tendrá el siguiente contenido:

- | |
|---|
| <ul style="list-style-type: none">1 Introducción<ul style="list-style-type: none">1.1 Propósito1.2 Ámbito del Sistema1.3 Definiciones, Acrónimos y Abreviaturas1.4 Referencias1.5 Visión general del documento2 Descripción General<ul style="list-style-type: none">2.1 Perspectiva del Producto2.2 Funciones del Producto2.3 Características de los usuarios2.4 Restricciones2.5 Suposiciones y Dependencias2.6 Requisitos Futuros3 Requisitos Específicos<ul style="list-style-type: none">3.1 Interfaces Externas3.2 Funciones3.3 Requisitos de Rendimiento3.4 Restricciones de Diseño3.5 Atributos del Sistema3.6 Otros Requisitos4 Apéndices5 Índice |
|---|

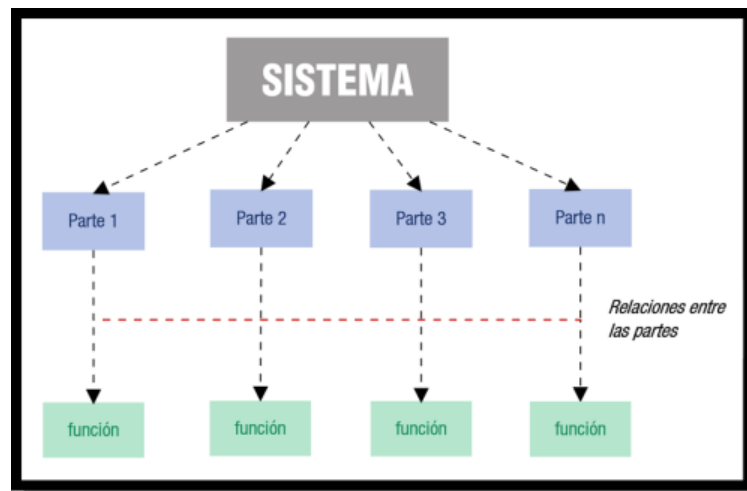
6.2. DISEÑO

Durante esta fase, donde ya sabemos lo que hay que hacer, el siguiente paso es ¿Cómo hacerlo?

Se debe dividir el sistema en partes y establecer qué relaciones habrá entre ellas.

Decidir qué hará exactamente cada parte.

En definitiva, debemos crear un modelo funcional-estructural de los requerimientos del sistema global, para poder dividirlo y afrontar las partes por separado.



En este punto, se deben tomar decisiones importantes, tales como:

- Entidades y relaciones de las bases de datos.
- Selección del lenguaje de programación que se va a utilizar.
- Selección del Sistema Gestor de Bases de Datos.
- Etc.

Los dos modelos de diseño más habituales son:

- **Diseño Estructurado:** Basado en el flujo de datos a través del sistema.
- **Diseño Orientado a Objetos:** El sistema se entiende como un conjunto de objetos que tienen propiedades y comportamientos, además de eventos que activan operaciones que modifican el estado de los objetos. Los objetos interactúan de manera formal con otros objetos.

Design is not just what it looks like and feels like. Design is how it works. ("El diseño no es sólo lo que parece y cómo parece. Diseño es cómo se trabaja"). Steve Jobs.

6.3. CODIFICACIÓN. TIPOS DE CÓDIGO

Durante la fase de codificación se realiza el proceso de programación.

Consiste en elegir un determinado lenguaje de programación, codificar toda la información anterior y llevarlo a código fuente.

Esta tarea la realiza el programador y tiene que cumplir exhaustivamente con todos los datos impuestos en el análisis y en el diseño de la aplicación.

Las características deseables de todo código son:

1. Modularidad: que esté dividido en trozos más pequeños.
2. Corrección: que haga lo que se le pide realmente.
3. Fácil de leer: para facilitar su desarrollo y mantenimiento futuro.
4. Eficiencia: que haga un buen uso de los recursos.
5. Portabilidad: que se pueda implementar en cualquier equipo.

Durante esta fase, el código pasa por diferentes estados:

- **Código Fuente:** es el escrito por los programadores en algún editor de texto. Se escribe usando algún lenguaje de programación de alto nivel y contiene el conjunto de instrucciones necesarias.

- **Código Objeto:** es el código binario resultado de compilar el código fuente.

La **compilación** es la traducción de una sola vez del programa, y se realiza utilizando un compilador. La **interpretación** es la traducción y ejecución simultánea del programa línea a línea.

El código objeto no es directamente inteligible por el ser humano, pero tampoco por la computadora. Es un código intermedio entre el código fuente y el ejecutable y sólo existe si el programa se compila, ya que si se interpreta (traducción línea a línea del código) se traduce y se ejecuta en un solo paso.

- **Código Ejecutable:** Es el código binario resultante de enlazar los archivos de código objeto con ciertas rutinas y bibliotecas necesarias. El sistema operativo será el encargado de cargar el código ejecutable en memoria RAM y proceder a ejecutarlo. También es conocido como **código máquina** y ya sí es directamente inteligible por la computadora.

Los programas interpretados no producen código objeto. El paso de fuente a ejecutable es directo.

6.3.1. FASES EN LA OBTENCIÓN DEL CÓDIGO

6.3.1.1. Código Fuente

El código fuente es el conjunto de instrucciones que la computadora deberá realizar, escritas por los programadores en algún lenguaje de alto nivel.

Este conjunto de instrucciones no es directamente ejecutable por la máquina, sino que deberá ser traducido al lenguaje máquina, que la computadora será capaz de entender y ejecutar.

Un aspecto muy importante en esta fase es la elaboración previa de un algoritmo, que lo definimos como un conjunto de pasos a seguir para obtener la solución del problema. El algoritmo lo diseñamos en pseudocódigo y con él, la codificación posterior a algún Lenguaje de Programación determinado será más rápida y directa.

Para obtener el código fuente de una aplicación informática:

1. Se debe partir de las etapas anteriores de análisis y diseño.
2. Se diseñará un algoritmo que simbolice los pasos a seguir para la resolución del problema.
3. Se elegirá una Lenguajes de Programación de alto nivel apropiado para las características del software que se quiere codificar.
4. Se procederá a la codificación del algoritmo antes diseñado.

La culminación de la obtención de código fuente es un documento con la codificación de todos los módulos (*Cada parte, con una funcionalidad concreta, en que se divide una aplicación*), funciones (*Parte de código muy pequeña con una finalidad muy concreta*), bibliotecas y procedimientos (*Igual que las funciones, pero al ejecutarse no devuelven ningún valor*) necesarios para codificar la aplicación.

Puesto que, como hemos dicho antes, este código no es inteligible por la máquina, habrá que **TRADUCIRLO**, obteniendo así un código equivalente pero ya traducido a código binario que se llama código objeto. Que no será directamente ejecutable por la computadora si éste ha sido compilado.

Un aspecto importante a tener en cuenta es su licencia. Así, en base a ella, podemos distinguir dos tipos de código fuente:

- Código fuente abierto. Es aquel que está disponible para que cualquier usuario pueda estudiarlo, modificarlo o reutilizarlo.
- Código fuente cerrado. Es aquel que no tenemos permiso para editarlo.

6.3.1.2. Código Objeto

El código objeto es un código intermedio.

Es el resultado de traducir código fuente a un código equivalente formado por unos y ceros que aún no puede ser ejecutado directamente por la computadora.

Es decir, es el código resultante de la compilación del código fuente.

Consiste en un bytecode (Código binario resultante de la traducción de código de alto nivel que aún no puede ser ejecutado.) que está distribuido en varios archivos, cada uno de los cuales corresponde a cada programa fuente compilado.

Sólo se genera código objeto una vez que el código fuente está libre de errores sintácticos y semánticos.

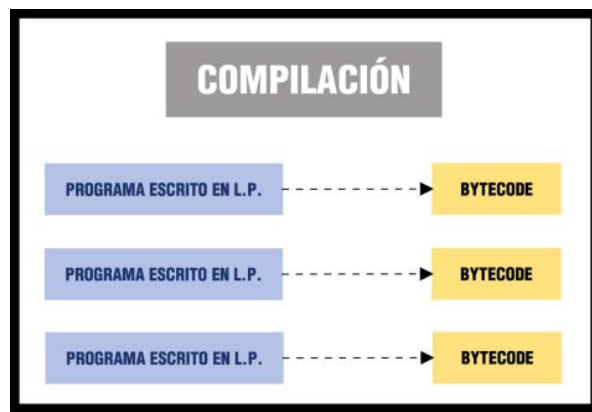
El proceso de traducción de código fuente a código objeto puede realizarse de dos formas:

- a) **Compilación:** El proceso de traducción se realiza sobre todo el código fuente, en un solo paso. Se crea código objeto que habrá que enlazar. El software responsable se llama

compilador (Software que traduce, de una sola vez, un programa escrito en un lenguaje de programación de alto nivel en su equivalente en lenguaje máquina.).

- b) **Interpretación:** El proceso de traducción del código fuente se realiza línea a línea y se ejecuta simultáneamente. No existe código objeto intermedio, es decir, no se genera ningún archivo intermedio, por lo que cada vez que se quiera ejecutar se volverá a interpretar (traducir). El software responsable se llama intérprete (Software que traduce, instrucción a instrucción, un programa escrito en un lenguaje de alto nivel en su equivalente en lenguaje máquina). El proceso de traducción es más lento que en el caso de la compilación, pero es recomendable cuando el programador es inexperto, ya que da la detección de errores es más detallada.

El código objeto es código binario, pero no puede ser ejecutado por la computadora.



6.3.1.3. Código Ejecutable

El código ejecutable, resultado de enlazar los archivos de código objeto, consta de un único archivo que puede ser directamente ejecutado por la computadora. No necesita ninguna aplicación externa. Este archivo es ejecutado y controlado por el sistema operativo.

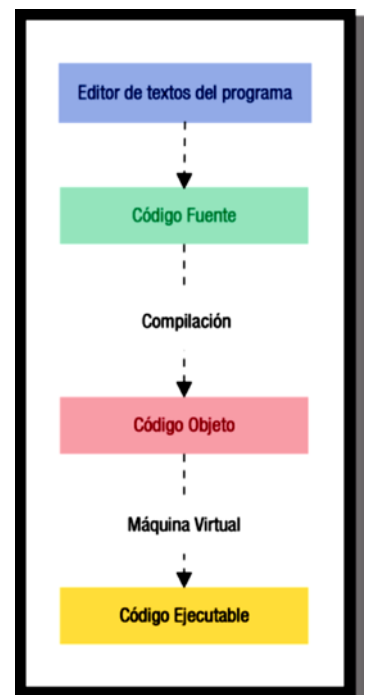
Para obtener un sólo archivo ejecutable, habrá que enlazar todos los archivos de código objeto, a través de un software llamado linker (Enlazador. Pequeño software encargado de unir archivos para generar un programa ejecutable.) y obtener así un único archivo que ya sí es ejecutable directamente por la computadora.

En el esquema de generación de código ejecutable, vemos el proceso completo para la generación de ejecutables.

A partir de un editor, escribimos el lenguaje fuente con algún lenguaje de programación.

A continuación, el código fuente se compila obteniendo código objeto o bytecode.

Este bytecode, a través de la máquina virtual, pasa a código máquina, ya directamente ejecutable por la computadora.



6.4. PRUEBAS

Una vez obtenido el software, la siguiente fase del ciclo de vida es la realización de pruebas. Normalmente, éstas se realizan sobre un conjunto de datos de prueba, que consisten en un conjunto seleccionado y predefinido de datos límite a los que las aplicaciones es sometida.

La realización de pruebas es imprescindible para asegurar la validación y verificación del software construido.

Entre todas las pruebas que se efectúan sobre el software podemos distinguir básicamente:

- **Pruebas unitarias:**
 - Consisten en probar, una a una, las diferentes partes del software y comprobar su funcionamiento (por separado, de manera independiente). JUnit es el entorno de pruebas para Java.
 -
- **Pruebas de integración:**
 - Se realizan una vez que se han realizado con éxito las pruebas unitarias y consistirán en comprobar el funcionamiento del sistema completo: con todas sus partes interrelacionadas.
 - La prueba final se denomina comúnmente Beta Test; ésta se realiza sobre el entorno de producción donde el software va a ser utilizado por el cliente (a ser posible, en los equipos del cliente y bajo un funcionamiento normal de su empresa).
 - El período de prueba será normalmente el pactado con el cliente.

6.5. DOCUMENTACIÓN

Todas las etapas en el desarrollo de software deben quedar perfectamente documentadas.

¿Por qué hay que documentar todas las fases del proyecto?

Para dar toda la información a los usuarios de nuestro software y poder acometer futuras revisiones del proyecto.

Tenemos que ir documentando el proyecto en todas las fases de este, para pasar de una a otra de forma clara y definida. Una correcta documentación permitirá la reutilización de parte de los programas en otras aplicaciones, siempre y cuando se desarrollen con diseño modular.

Distinguimos tres grandes documentos en el desarrollo de software:

	Documentos a elaborar en el proceso de desarrollo de software		
	GUÍA TÉCNICA	GUÍA DE USO	GUÍA DE INSTALACIÓN
Quedan reflejados:	- El diseño de la aplicación. - La codificación de los programas. - Las pruebas realizadas.	- Descripción de la funcionalidad de la aplicación. - Forma de comenzar a ejecutar la aplicación. - Ejemplos de uso del programa. - Requerimientos software de la aplicación. - Solución de los posibles problemas que se pueden presentar.	Toda la información necesaria para: - Puesta en marcha. - Explotación. - Seguridad del sistema.
¿A quién va dirigido?	Al personal técnico en informática (analistas y programadores).	A los usuarios que van a usar la aplicación (clientes).	Al personal informático responsable de la instalación, en colaboración con los usuarios que van a usar la aplicación (clientes).
¿Cuál es su objetivo?	Facilitar un correcto desarrollo, realizar correcciones en los programas y permitir un mantenimiento futuro.	Dar a los usuarios finales toda la información necesaria para utilizar la aplicación	Dar toda la información necesaria para garantizar que la implantación de la aplicación se realice de forma segura, confiable y precisa.

6.6. EXPLOTACIÓN

Después de todas las fases anteriores, una vez que las pruebas nos demuestran que el software es fiable, carece de errores y hemos documentado todas las fases, el siguiente paso es la explotación.

Aunque diversos autores consideran la explotación y el mantenimiento como la misma etapa, nosotros vamos a diferenciarlas en base al momento en que se realizan.

La explotación es la fase en que los usuarios finales conocen la aplicación y comienzan a utilizarla.

La explotación es la instalación, puesta a punto y funcionamiento de la aplicación en el equipo final del cliente.

En el proceso de instalación, los programas son transferidos al computador del usuario cliente y posteriormente configurados y verificados.

Es recomendable que los futuros clientes estén presentes en este momento e irles comentando cómo se va planteando la instalación.

En este momento, se suelen llevar a cabo las Beta Test, que son las últimas pruebas que se realizan en los propios equipos del cliente y bajo cargas normales de trabajo.

Una vez instalada, pasamos a la fase de configuración.

En ella, asignamos los parámetros de funcionamiento normal de la empresa y probamos que la aplicación es operativa. También puede ocurrir que la configuración la realicen los propios usuarios finales, siempre y cuando les hayamos dado previamente la guía de instalación. Y también, si la aplicación es más sencilla, podemos programar la configuración de manera que se realice automáticamente tras instalarla. (Si el software es "a medida", lo más aconsejable es que la hagan aquellos que la han fabricado).

Una vez se ha configurado, el siguiente y último paso es la fase de producción normal. La aplicación pasa a manos de los usuarios finales y se da comienzo a la explotación del software.

Es muy importante tenerlo todo preparado antes de presentarle el producto al cliente: será el momento crítico del proyecto.

6.7. MANTENIMIENTO

Sería lógico pensar que con la entrega de nuestra aplicación (la instalación y configuración de nuestro proyecto en los equipos del cliente) hemos terminado nuestro trabajo.

En cualquier otro sector laboral esto es así, pero el caso de la construcción de software es muy diferente.

La etapa de mantenimiento es la más larga de todo el ciclo de vida del software.

Por su naturaleza, el software es cambiante y deberá actualizarse y evolucionar con el tiempo. Deberá ir adaptándose de forma paralela a las mejoras del hardware en el mercado y afrontar situaciones nuevas que no existían cuando el software se construyó.

Además, siempre surgen errores que habrá que ir corrigiendo y nuevas versiones del producto mejores que las anteriores.

Por todo ello, se pacta con el cliente un servicio de mantenimiento de la aplicación (que también tendrá un coste temporal y económico).

El mantenimiento se define como el proceso de control, mejora y optimización del software.

Su duración es la mayor en todo el ciclo de vida del software, ya que también comprende las actualizaciones y evoluciones futuras del mismo.

Los tipos de cambios que hacen necesario el mantenimiento del software son los siguientes:

- **Perfectivos:** Para mejorar la funcionalidad del software.
- **Evolutivos:** El cliente tendrá en el futuro nuevas necesidades. Por tanto, serán necesarias modificaciones, expansiones o eliminaciones de código.
- **Adaptativos:** Modificaciones, actualizaciones... para adaptarse a las nuevas tendencias del mercado, a nuevos componentes hardware, etc.
- **Correctivos:** La aplicación tendrá errores en el futuro (sería utópico pensar lo contrario).

7. METODOLOGÍAS DE DESARROLLO DE SOFTWARE

«Una metodología es un conjunto de filosofías, fases, procedimientos, reglas, técnicas, herramientas, documentación y aspectos de formación para los desarrolladores de sistemas de información».

Maddison (1983)

Entonces, una metodología puede seguir uno o varios modelos de ciclo de vida, esto es, el ciclo de vida indica qué es lo que hay obtener a lo largo del desarrollo del proyecto, pero no cómo, que es lo que indicará la metodología. Por tanto, el ciclo de vida es el modelo general que se sigue para el desarrollo de software, mientras que una metodología establece las tareas concretas que hay que llevar a cabo, cómo se debe llevar a cabo cada tarea, con qué herramientas, ...

Como hemos visto, el proceso de desarrollo de software ha de dividirse en una serie de fases (en las que se deben llevar a cabo una serie de tareas) para que sea manejable.

Para llevar a cabo dichas tareas, se siguen **procedimientos** establecidos y se obtienen unos **productos** que serán *intermedios* (si son la base para otra tarea posterior) o *finales* (entregables al cliente).

Para aplicar los procedimientos, se usan una o varios métodos o **técnicas** (diagramas, gráficos, documentos, ...) que pueden o no hacer uso de determinadas **herramientas**.

Las herramientas que permiten automatizar las tareas de desarrollo de software reciben el nombre de herramientas CASE (Computer Aided Software Engineering).

7.1. Evolución de las metodologías

Se definen tres periodos en la evolución de las metodologías de desarrollo de software:

1. **Desarrollo convencional:** durante los primeros años del desarrollo de software las prácticas de desarrollo eran totalmente artesanales y no se seguía ninguna metodología. Esto provocaba muchos problemas y desencadenó lo que se llamó la *crisis del software*.
2. **Metodologías estructuradas:** la primera respuesta a esta crisis fue la regulación de la tarea de programación con la difusión de la programación estructurada, a la que siguió el surgimiento de métodos para el diseño y análisis estructurado, dando lugar a las metodologías estructuradas, que abarcan la totalidad del ciclo de vida del software. Recordemos que en la metodología estructurada, las aplicaciones se conciben como programas compuestos por diversos módulos reutilizables entre los que se realizan llamadas.
3. **Metodologías orientadas a objetos:** en la década de los 80 surgieron los lenguajes orientados a objetos y más tarde, métodos para el diseño y análisis orientados a objetos. En esta metodología, una aplicación se compone de varios objetos que tienen un estado (atributos) y un comportamiento (métodos).

En la actualidad, la metodología de referencia es el proceso unificado de desarrollo de software impulsada por Ivar Jacobson, Grady Booch y James Rumbaugh, autores del lenguaje unificado de modelado (**UML**) que estudiaremos en los temas siguientes. Es un lenguaje gráfico que siguiendo unas normas permite visualizar, construir, especificar y documentar un sistema. Define los diagramas que representan diferentes aspectos de un sistema siguiendo el paradigma orientado a objetos.

Recientemente, se han desarrollado los modelos de **desarrollo ágil**, que son metodologías de desarrollo que intentan simplificar las metodologías existentes, intentando que el desarrollo no sea excesivamente burocrático pero que tampoco esté organizado.

7.2. Modelo de desarrollo ágil

En este tipo de metodología se le da mucha importancia a la entrega de software que funcione al cliente, cada poco tiempo y no tanta al rigor en la ingeniería del software y en los productos intermedios de cada fase. Uno de los objetivos es la entrega rápida de software incremental haciendo uso de equipos de desarrollo pequeños y motivados.

Esto hace que haya una mayor facilidad para incorporar cambios a lo largo del desarrollo a bajo coste ya que, cuanto más tarde en el proceso de desarrollo se detecta la necesidad de realizar un cambio, más costoso es incorporarlo. También resultará más costoso si este cambio afecta a etapas iniciales (análisis o diseño). En esta metodología, gracias a las entregas cada poco tiempo y al feedback del cliente en cada entrega, se consiguen reducir el coste de los cambios.



Los doce principios establecidos por esta metodología son:

1. La máxima prioridad es satisfacer al cliente a través de la entrega rápida y continua de software valioso.
2. Los requisitos cambiantes son bienvenidos, aun en etapas avanzadas del desarrollo, pues, cuando hay cambios, los procesos ágiles son beneficiosos para la ventaja competitiva del cliente.
3. Se realizan entregas frecuentes y lo más pronto que se pueda de software que funcione, preferiblemente entre dos semanas y un par de meses.

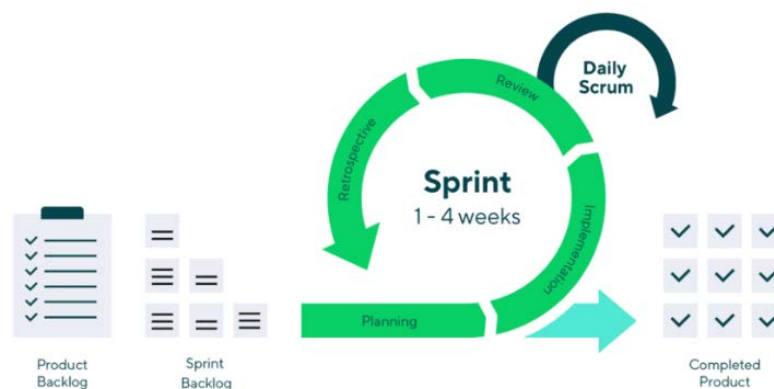
4. Las personas responsables del negocio y las encargadas del desarrollo del software trabajan conjuntamente, a diario y durante todo el proyecto.
5. Los proyectos son desarrollados por individuos motivados. Debe proporcionarse a estos el entorno y el apoyo adecuados, y confiar en que harán el trabajo.
6. El método más eficiente y eficaz para comunicar información al equipo de desarrollo y entre sus miembros es la conversación cara a cara.
7. La principal medida para progresar es un software que funcione.
8. Los procesos ágiles impulsan el desarrollo sostenible. Los promotores, las personas que desarrollan el software y las usuarias de este deben ser capaces de mantener un ritmo constante de forma indefinida.
9. La atención continua a la excelencia técnica y al buen diseño mejoran la agilidad.
10. Es esencial la simplicidad: el arte de maximizar la cantidad de trabajo no realizado.
11. Las mejores arquitecturas, requisitos y diseños surgen de los equipos con organización propia.
12. El equipo reflexiona con regularidad sobre cómo ser más eficaz, para después perfeccionar y ajustar su comportamiento en consecuencia.

SCRUM

Es un modelo de desarrollo **ágil** tiene como objetivo la entrega de valor (productos) al cliente en cortos periodos de tiempo y se basa en tres pilares:

1. Transparencia: todas las personas involucradas en el proyecto conocen en todo momento qué ocurre y cómo, lo que permite que haya una visión global del proyecto.
2. Inspección: todos los miembros del equipo inspeccionan de manera autoorganizada el progreso del proyecto para detectar posibles problemas.
3. Adaptación: cuando es necesario realizar algún cambio en el proyecto, el equipo se adapta para conseguir el objetivo.

Esta metodología se organiza en **actividades estructurales**: requerimientos, análisis, diseño, evolución y entrega. Todas estas actividades se organizan en **sprints**.



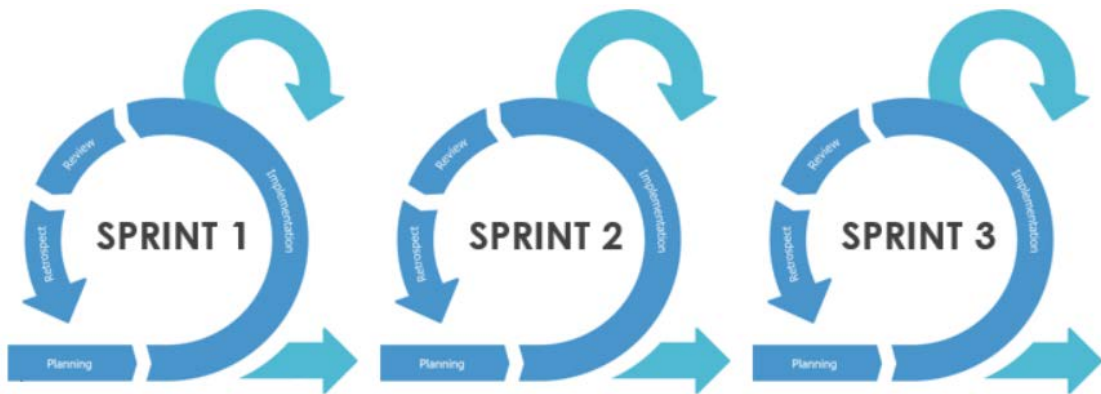
Un **sprint** es el periodo de tiempo que se emplea para realizar el trabajo necesario para entregar valor al cliente. La duración máxima es de un mes, aunque es aconsejable ajustarse más a un plazo de dos semanas. Se organiza en cuatro fases.

Antes de iniciarse un sprint, se realiza la reunión de **sprint planning**, donde todo el equipo establece qué tareas se van a realizar (sprint backlog) en ese sprint, cómo se van a realizar y el objetivo del mismo. Al final del sprint, se hace una entrega de valor (producto increment) al cliente.

Dentro de cada sprint, se organizan reuniones diarias con una duración máxima de 15 minutos. En las reuniones, se responde a las siguientes preguntas: ¿qué hicimos ayer?, ¿qué vamos a hacer hoy? Y ¿tenemos algún problema que hay que solucionar?

Una vez termina el sprint, se entrega el producto al cliente (**sprint review**). El equipo de desarrollo muestra el funcionamiento del producto al cliente. Éste tendrá que validarlo. Además, durante esta reunión, el cliente proporciona feedback útil para nuevas tareas. Después el equipo hará una **retrospectiva**, que es un análisis del trabajo realizado durante el sprint, proponiendo mejoras para el siguiente sprint.

Finalizada esta reunión, se comienza el siguiente sprint.



Para más información: <https://www.youtube.com/watch?v=HhC75lonpOU>

8. HERRAMIENTAS UTILIZADAS EN PROGRAMACIÓN

Para llevar a cabo la codificación y prueba de los programas se suelen utilizar entornos de programación. Estos entornos nos permiten realizar diferentes tareas:

- Crear, editar y modificar el código fuente del programa.
- Compilar, montar y ejecutar el programa.
- Examinar el código fuente.
- Ejecutar el programa en modo depuración.
- Realizar pruebas del programa de forma automática.
- Generar documentación.
- Gestionar los cambios que se van haciendo en el programa (control de versiones).
- Etc.

A estos entornos de programación se les suele llamar entornos de desarrollo integrado o **IDE** (Integrated Development Environment). Los IDEs están diseñados para maximizar la productividad del programador. Un IDE es un programa informático formado por un conjunto de herramientas de programación que facilitan las tareas de creación, modificación, compilación, implementación y depuración de software.

La mayoría de los IDEs actuales proporcionan un entorno de trabajo visual formado por ventanas, barras de menús, barras de herramientas, paneles laterales para presentar la estructura en árbol de los proyectos o del código del programa que estamos editando, pestañas adicionales con el resultado de aplicar determinadas herramientas o bien que nos permiten tener abiertos varios ficheros del proyecto, asistentes que ayudan al programador a desarrollar parte del proyecto o del código, etc.

Los editores suelen ofrecer facilidades como el resaltado de la sintaxis utilizando diferentes colores y tipos de letra, el emparejamiento de llaves o paréntesis poniendo el cursor sobre uno de ellos, el plegado y desplegado de código, por ejemplo, una función se puede reducir a una simple línea de cabecera.

Un mismo IDE puede funcionar con varios lenguajes de programación, este es el caso de Eclipse o Netbeans que mediante la instalación de plugins se le puede añadir soporte de lenguajes adicionales.